

Laboratorio 1: Pipeline de preprocesamiento para datos secuenciales

Proyecto: clasificación de sentimiento en reseñas de usuarios

Dataset: Amazon Fine Food Reviews (Kaggle)

1. Objetivo

El objetivo de este laboratorio fue construir un pipeline reproducible que convierta reseñas escritas por usuarios en secuencias numéricas de longitud fija, listas para entrenar un modelo secuencial de clasificación de sentimiento. En esta unidad no se entrena ningún modelo: el entregable es el dato preparado y la trazabilidad de cómo se preparó.

2. Dataset y pertinencia

Se trabajó con Amazon Fine Food Reviews (McAuley y Leskovec, 2013): 568.454 reseñas de productos alimenticios publicadas entre octubre de 1999 y octubre de 2012, con 10 variables. De todas ellas, sólo tres entran al pipeline:

- **Text:** el texto de la reseña. Es la entrada del modelo.
- **Score:** la calificación de 1 a 5. No entra como predictor; solo se usa para derivar la etiqueta.
- **Time:** la marca temporal. Define cómo se parten los datos, no es una característica del modelo.

Se eligieron estas 3 columnas de las 10 totales ya que son las necesarias que se alinea con el objetivo, y es que entre las demás variables encontramos identificadores únicos que puede que al entrenamiento aporte información, pero nuestro fin es realizar un clasificador de sentimiento en base solamente la opinión del usuario (la reseña). Para más información de las demás variables se puede encontrar en el código adjunto a la entrega o buscando en Kaggle el nombre del dataset.

3. Decisiones técnicas

3.1 Valores faltantes

Los únicos nulos del dataset están en ProfileName (26) y Summary (27), y ninguna de las dos variables entra al modelo. Text, Score y Time están completos, y Score no tiene valores fuera del rango 1–5. No se eliminó ninguna fila por este motivo: descartar registros completos por un campo que no se usa habría sido una pérdida incoherente de datos.

3.2 Duplicados:

En este apartado se encontraron que 174.875 filas repiten el texto de otra reseña. Al analizarlas encontramos que la enorme mayoría de estos duplicados corresponden a un grupo que comparte las siguientes características: (Mismo usuario, misma fecha y la misma calificación, pero con distinto ProductId). Tras investigar la razón de este comportamiento, encontramos que esto es típico de datasets extraídos de Amazon: cuando un producto tiene variantes (sabores, tamaños, presentaciones), la misma reseña aparece en todas. Para facilitar más la comprensión de este comportamiento, pensemos sobre un producto que se vende en Amazon, una carcasa para una marca, en la misma página del producto podemos encontrar otras opciones, sea para otra línea del celular o sea que la carcasa es de otro color. Un usuario que compra una carcasa de color rojo y realiza una reseña sobre esa carcasa, esa reseña va a duplicarse y salir en las reseñas de la carcasa de color Azul, Café, Negro, etc...

En estos casos se eliminaron todos los duplicados que comparten esas características pero se conservó la ocurrencia más antigua, que es la del producto que el usuario efectivamente compró. Aparte de eso se detectaron 428 filas donde el mismo texto aparecía con calificaciones distintas; se aplicó el mismo criterio, porque entrenar con la misma frase etiquetada de dos maneras es enseñarle al modelo a contradecirse.

En total se eliminaron 175.335 registros (30,8 % del original) y el corpus quedó en 393.119 reseñas útiles. Esta depuración se hizo antes de partir los datos, para que una misma reseña no pudiera quedar al mismo tiempo en entrenamiento y en prueba.

3.3 Normalización del texto

Sobre una columna nueva, Text_clean, se decodificaron las entidades HTML (&, "), se eliminaron las etiquetas de marcado (
), se sustituyeron las URLs, se pasó todo a minúsculas y se normalizaron los espacios. El texto original se conservó intacto para poder auditar en cualquier momento el efecto de la transformación.

Deliberadamente no se aplicó stemming, lematización ni eliminación de stopwords. En clasificación de sentimiento, palabras como not, but o very son justamente las que cambian el significado de una frase: quitarlas equivale a borrar el "no" de "no me gustó" cambiando completamente el significado de la reseña ya que se elimina la negación y al revisar con el score se genera una contradicción.

3.4 Variable objetivo y desbalance

La etiqueta Sentimiento se derivó de Score con la regla (1 o 2 = Negativo), (3 = Neutral), (4 o 5 = Positivo), y se codificó como 0, 1 y 2 respectivamente. La distribución resultante es claramente desbalanceada: cerca del 78 % de las reseñas es Positivo, 14 % Negativo y 8 % Neutral. Es prácticamente la misma proporción del dataset original, lo que confirma que la limpieza no distorsionó ni introdujo sesgos.

El desbalance se documenta pero no se corrige en esta etapa. Cualquier ponderación de clases o remuestreo es una decisión de entrenamiento y debe aplicarse solo sobre el conjunto de entrenamiento; hacerlo antes de partir los datos contaminaría la evaluación y daría métricas optimistas.

3.5 Partición Train / Validation / Test

Se usó un corte cronológico 70/15/15 sobre los datos ordenados por fecha: las reseñas más antiguas entrenan, las intermedias validan y las más recientes prueban. Una partición aleatoria dejaría al modelo entrenando con reseñas de 2012 para predecir reseñas de 2009, algo que en producción nunca ocurre, porque allí siempre se predice hacia adelante. Con 393.119 registros, un 15 % equivale a cerca de 59.000 reseñas por conjunto, tamaño suficiente para medir el desempeño de forma estable incluso en la clase minoritaria, pero para validar correctamente esta teoría se debe de validar en su momento al entrenar y rectificar cualquier inconveniente que surja.

3.6 Valores atípicos

La única variable continua que llega al modelo es la longitud de la secuencia; los atípicos de las columnas de utilidad (Helpfulness) quedan fuera del alcance porque esas columnas no son parte de la entrada. Con el criterio del rango intercuartílico aparecen reseñas muy por encima del rango habitual, y el máximo supera los 2.500 tokens.

Aun así no se eliminaron. Una reseña larga no es un error de registro sino una opinión detallada, y suele ser la que más señal de sentimiento contiene; borrarlas sesgaría el corpus hacia los textos cortos. El costo que generan se controla más adelante con el truncamiento, que acota la longitud sin eliminar ninguna fila.

3.7 Vocabulario

El índice de palabras se construyó únicamente con el conjunto de entrenamiento (103.517 palabras distintas) y se aplicó sin cambios a validación y prueba. Construirlo con el corpus completo sería como revisar el examen antes de presentarlo: el modelo conocería de antemano el vocabulario de los periodos futuros y el desempeño quedaría sobreestimado.

Se fijó el vocabulario en 20.000 palabras, que cubren el 99,07 % de los tokens de entrenamiento. Ampliarlo a 50.000 solo agrega 0,66 puntos de cobertura a cambio de 2,5 veces más parámetros en la futura capa de embedding, y reducirlo a 5.000 dejaría fuera cerca del 4,5 % de los tokens, una proporción ya relevante en

textos cortos. Las dos primeras posiciones del índice se reservaron para <PAD> (rellena oraciones cortas para igualar el tamaño) y <OOV> (reemplaza palabras que el modelo no conoce).

3.8 Longitud, padding y truncamiento

Se fijó MAX_LEN = 200 tokens, un valor situado entre el percentil 90 (157 tokens) y el percentil 95 (211) del conjunto de entrenamiento. Con 200 se trunca el 5,68 % de las reseñas; con 100 se perdería el 23,48 %, y con 300 el truncamiento bajaría apenas al 1,88 % pero el relleno subiría al 74,56 % de la matriz, de igual forma, estos ajustes son configuraciones iniciales que en la etapa de entrenamiento se van a evaluar si mantener o cambiar por otros valores.

Para comprender mejor lo que explica el texto anterior imaginen lo siguiente: El problema de la talla única: a las reseñas largas la camiseta les queda corta y se pierde el final, y a las cortas les sobra tela. La diferencia está en que el truncamiento borra información de forma irreversible, mientras que el relleno se puede neutralizar dentro del modelo con enmascaramiento (mask_zero=True). Por eso la decisión se inclina hacia tolerar relleno antes que perder texto.

4. Resultados

Tabla de datos generales de las transformaciones

Elemento	Resultado
Dataset original (registros)	568.454
Registros eliminados (duplicados y conflictos)	175.335
Registros después de limpieza	393.119
Train	275.183 (70,0 %)
Validation	58.968 (15,0 %)
Test	58.968 (15,0 %)
Número de clases	3 (Negativo, Neutral, Positivo)
Vocabulario (palabras + PAD + OOV)	20.000 + 2 = 20.002
Cobertura del vocabulario en Train	99,07 %
MAX_LEN	200 tokens
Dimensión X_train	(275.183, 200)
Dimensión X_val	(58.968, 200)
Dimensión X_test	(58.968, 200)

Tabla desbalanceo de clases en cada etapa para train-validation-test

Clase	Train (%)	Validation (%)	Test (%)
Negativo	13,82	16,34	15,77
Neutral	7,38	8,22	7,75
Positivo	78,80	75,45	76,48

Tabla calidad de las secuencias para cada grupo train-validation-test

Conjunto	Reseñas	Truncadas (%)	Tokens OOV (%)
Train	275.183	5,68	0,82
Validation	58.968	6,96	0,89
Test	58.968	5,73	0,94

La tasa de palabras desconocidas se mantiene por debajo del 1 % en los tres conjuntos, lo que indica que un vocabulario aprendido solo con datos antiguos sigue cubriendo bien el lenguaje de los periodos posteriores.

Tabla de los cortes temporales para cada grupo train-validation-test

Conjunto	Reseñas	Desde	Hasta
Train	275.183	08-10-1999	14-02-2012
Validation	58.968	14-02-2012	30-06-2012
Test	58.968	30-06-2012	26-10-2012

5. Validación de consistencia

Antes de dar por cerrado el pipeline se ejecutaron 18 comprobaciones automáticas, todas superadas:

- cada matriz X tiene tantas filas como su vector y
- todas las secuencias miden exactamente 200 posiciones
- ningún índice se sale del rango del vocabulario
- ninguna secuencia quedó completamente vacía
- las etiquetas solo toman los valores 0, 1 y 2
- no hay solapamiento temporal entre entrenamiento, validación y prueba
- la suma de los tres conjuntos coincide con el dataset limpio

Sobre fuga de información se aplicaron tres controles: el vocabulario y los parámetros (tamaño de vocabulario y MAX_LEN) se estimaron solo con entrenamiento; la deduplicación se hizo antes de partir los datos; y Score no participa en la construcción de las secuencias, solo origina la etiqueta. Adicionalmente se dejaron tres ejemplos completos del recorrido texto original → texto limpio → tokens → secuencia → etiqueta, para poder auditar el pipeline a nivel de una sola reseña.

6. Limitaciones

1. **Truncamiento:** Cerca del 6 % de las reseñas pierden su parte final. El efecto real de esa pérdida sólo podrá medirse cuando exista un modelo entrenado.
2. **Relleno:** Alrededor del 63 % de las celdas de la matriz es padding. Deberá neutralizarse con enmascaramiento en la capa de embedding o agrupando las secuencias por longitud durante el entrenamiento.
3. **Desbalance sin tratar:** La clase Neutral no alcanza el 8 % de los datos. La estrategia de compensación corresponde a la etapa de modelado.
4. **Tokenización por palabra:** No captura morfología ni subpalabras, así que los términos poco frecuentes caen en <OOV>. Una tokenización por subpalabras reduciría esa pérdida a cambio de más complejidad.
5. **Alcance del corpus:** Son reseñas en inglés de productos alimenticios entre 1999 y 2012; las conclusiones no se extrapolan a otros dominios, idiomas o periodos.

7. Reproducibilidad

El notebook se ejecuta de principio a fin sin intervención manual. Se fijó la semilla 42 y se registraron las versiones de Python, pandas, NumPy y matplotlib usadas en la ejecución del documento. Se persistieron cuatro artefactos: `secuencias.npz` (las matrices X e y de los tres conjuntos), `reviews_clean.csv` (trazabilidad entre texto y etiqueta), `word_index.json` (el vocabulario) y `parametros_pipeline.json` (todos los parámetros del pipeline). Con esos archivos, la etapa de modelado puede consumir las secuencias sin volver a ejecutar el preprocesamiento, y cualquier reseña nueva puede transformarse exactamente con el mismo vocabulario y los mismos parámetros.

8. Repositorio

Para visualizar las gráficas, código o en general contar con un seguimiento en general del proyecto, puedes encontrar el siguiente repositorio donde estaremos actualizando en un futuro los avances.

Link repositorio:

https://github.com/Kinanel07/Clasificador_sentimientos_LAB1_MSECDATOSTEMPORALES

8. Uso de IA

Se implementó IA para la generación de código para el pipeline, cabe recalcar que cada celda fue validada, cada transformación se analizó en conjunto del grupo, se implementó el uso de IA para agilizar la creación del pipeline, pero no se usó de esta para conclusiones y transformaciones no antes validadas que busquen cumplir con el objetivo.

Referencias

Brownlee, J. (2020). Deep learning for natural language processing. Machine Learning Mastery.

Chollet, F. (2021). Deep learning with Python (2.^a ed.). Manning Publications.

Jurafsky, D., y Martin, J. H. (2024). Speech and language processing (3.^a ed., borrador). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>

McAuley, J., y Leskovec, J. (2013). From amateurs to connoisseurs: Modeling the evolution of user expertise through online reviews. Proceedings of the 22nd International Conference on World Wide Web, 897–908.

McAuley, J., y Leskovec, J. (2013). Amazon Fine Food Reviews [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews>